AI 388 / CS 388 Natural Language Processing

Module 7: POS Tagging, HMMs, Parsing

Topics 19, 20, 21, 22

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Full derivations** (added Oct 20): every formula derived step by step with nothing assumed, plus low-dimensional by-hand examples of the algorithms worked in lecture and "conceptual checks" for Viterbi/CKY.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide **1**

19 POS Tagging and Its Ambiguities **3**

19.1 Tags (lecture crash course) . . . 3
19.2 The "Fed raises interest rates 0.5 percent" example . . . 3
19.3 Tagging with classifiers (lecture 7.1.0) . . . 4
19.4 Exercises . . . 4
19.5 Practice . . . 5

20 Hidden Markov Models: Definition, Estimation, Viterbi **6**

20.1 Definition . . . 6
20.2 Parameter estimation: count and normalize . . . 7
20.3 Inference: Viterbi . . . 7
20.4 Full derivations . . . 8
20.5 HMMs for POS tagging in practice . . . 9
20.6 Practice . . . 10

21 Constituency Syntax: Trees and Ambiguity **11**

21.1 What trees look like . . . 11
21.2 Ambiguities (must recognize) . . . 12
21.3 Constituency tests (how linguists justify phrases) . . . 12
21.4 Practice . . . 13

22 PCFGs, Binarization, Markovization, CKY **14**

22.1 CFG and PCFG . . . 14
22.2 From treebank to grammar . . . 15
22.3 Binarization . . . 15
22.4 Markovization (refining grammars) . . . 15
22.5 CKY . . . 16
22.6 Full derivations and conceptual checks . . . 17
22.7 Practice . . . 18

TOPIC 19

POS Tagging and Its Ambiguities

Sources: Lectures 7.0.0 Part-of-Speech Tagging, 7.1.0 Sequence Labeling, Tagging with Classifiers, 7.3.1 HMMs for POS Tagging; slides seg-22–seg-26; Exercises 7.0, 7.1.

In plain words

POS tagging assigns each word a grammatical category (noun, verb, . . .). Many words can take several tags (*record*, *arms*, *raises*), and the right one depends on context. The number of possible tag sequences grows multiplicatively, so even short sentences have dozens of readings, some grammatical but absurd. Classifiers that tag each word independently can produce incoherent sequences. Sequence models (HMMs, CRFs) score whole sequences. **You won't be asked to know tag definitions**; they'll be provided. You must reason about ambiguity and count/evaluate readings.

19.1 Tags (lecture crash course)

- **Open class** (new words join): nouns (proper: IBM, Italy; common: cat, snow), verbs, adjectives, adverbs (swiftly).
- **Closed class** (function words): determiners (the, some), conjunctions (and, or), pronouns, auxiliaries (*had* gone), modals (could), prepositions (up a mountain), particles (made *up* the story; verb–particle construction).
- Penn Treebank: 45 fine-grained tags, e.g. NNP (proper noun), NNS (plural noun), VBD (past), VBN (participle), VBZ (3rd-singular present), VBP (non-3rd present), VB (base form).

Why tag? Pronunciation (*REcord* vs. *reCORD*), information extraction (arms as noun vs. verb), and a building block for parsing.

19.2 The "Fed raises interest rates 0.5 percent" example

Worked example: counting readings

Fed	raises	interest	rates	0.5	percent
NNP, VBD, VBN	NNS, VBZ	NN, VBP, VB	NNS, VBZ	CD	NN

Number of tag sequences $= 3 \times 2 \times 3 \times 2 \times 1 \times 1 =$ **36**.
Intended: Fed/NNP raises/VBZ interest/NN rates/NNS ("the Fed raises [interest rates]"). The noun–noun compound *interest rates* is fine; *interest* is not an adjective.
Syntactically valid but absurd: Fed/NNP raises/NNS interest/VBP rates/NNS: "[Fed raises] interest [the rates] 0.5 percent" (the raises make the rates interested). Syntax permits it; only world knowledge (rates are inanimate) rules it out. That's what makes tagging and parsing hard for machines.

19.3 Tagging with classifiers (lecture 7.1.0)

- Sequence labeling: input $\mathbf{x} = (x_1, \ldots, x_n)$, output $\mathbf{y} = (y_1, \ldots, y_n)$, same length. This is **structured** classification.
- Independent LR per position: $P(y_i = y \mid \mathbf{x}, i)$. Features must depend on the **position** i. Whole-sentence BoW features are identical for every position, so every word would get the same tag distribution.
- Good features (DF-style indicators): CurrWord=*interest* & tag=NN; PrevWord=*raises*; NextWord=*rates*; prefixes/suffixes (-ing, -ed, -s); capitalization; digits; word length; seen-before?
- **Problem:** independent predictions can be *incoherent* (NNS followed by VBN, etc.). Sequence models fix this: **HMMs** (generative, Topic 20) and CRFs (discriminative; not on the current exam).
- Feedforward tagger (Botha et al.): concatenate embeddings of previous/current/next word (+ character n-grams). Strong across many languages.

Accuracy facts (Penn Treebank)

Most-frequent-tag baseline about 90% (many unambiguous words: *the*, punctuation). Trigram HMM about 95% (55% on unknown words). TnT (Brants) slightly higher. MaxEnt tagger (Toutanova & Manning) about 97%; BiLSTM-CRF/BERT about 97.5%. Manning (2011): of the remaining errors, about 60% are underspecified, unclear, or gold errors. Common confusions: JJ/NN (*official* knowledge), RP/IN (made *up*), VBD/VBN (recently *sold* shares).

19.4 Exercises

Canvas Exercise 7.0

Tags: Noun, Verb, Adjective, Adverb, Determiner, Conjunction, Pronoun, Preposition, Particle, Cardinal. Sentence "I make small gains on my investments". Possible tags for *gains* (any context):

Answer: Noun, Verb. Most likely here:

Answer: Noun. Possible for *make*:

Answer: Noun, Verb ("the make of the car"). Most likely full tagging:

Answer: N V ADJ N Prep N N. (The answer key tags *I* and *my* as N in these options.)

Canvas Exercise 7.1

Classify *interest* in "Fed raises interest rates 0.5 percent" as noun/verb. Does a sentence-level unigram BoW work?

Answer: No: it doesn't capture the position of *interest* or its neighbors. Effective features:

Answer: Indicator(CurrWord="interest"). Not Contains("interest"), ThirdWord, or CurrWord=Noun. Other useful information:

Answer: prefixes/suffixes, context words, seen before or not, word length, contains digits (all of them).

Traps

- Count readings by multiplying the number of possible tags per word.
- "Possible tag" means in *some* context; "most likely" means in *this* context.
- Noun–noun compounds are normal (interest/NN rates/NNS). Don't force adjectives.
- Tagging features must be position-specific.

19.5 Practice

Practice

1. "Time flies like an arrow": give two readings with different tags for *flies*/*like*. Tag options: time {N, V}, flies {N, V}, like {V, P}, an {D}, arrow {N}. How many tag sequences?
2. Why does a per-word classifier with only CurrWord features get about 90% but fail on "can" in "they can fish"?

Answers

1. time/N flies/V like/P an/D arrow/N (normal); time/N flies/N like/V an arrow ("time-flies enjoy an arrow"). $2 \cdot 2 \cdot 2 \cdot 1 \cdot 1 = 8$ sequences.
2. CurrWord only gives each word type one most-likely tag. *can* as modal/verb vs. noun needs context (previous word "they").

TOPIC 20

Hidden Markov Models: Definition, Estimation, Viterbi

Sources: Lectures 7.2.0 Hidden Markov Models, 7.2.1 HMMs: Parameter Estimation, 7.3.0 HMMs: Viterbi Algorithm, 7.3.1 HMMs for POS Tagging; slides seg-24–seg-26; Exercises 7.2, 7.3; Fall 2022 Q1(2), Q1(8–10); Fall 2023 Q1(19–21). **De-emphasized this year (especially Viterbi), but in scope.**

In plain words

An HMM tells a *generative story*: pick a first tag, emit a word from it, move to a next tag (depending only on the current tag), emit a word, . . . , then STOP. **Parameters** = three tables of probabilities (initial, transition, emission), estimated by **counting and normalizing** tagged data. **Tagging** = find the most probable tag sequence for a sentence. **Viterbi** dynamic programming does it in $O(nk^2)$ instead of checking all k^n sequences.

20.1 Definition

HMM joint probability

Tags $y_i \in \mathcal{T}$, words $x_i \in V$.

$$P(\mathbf{y}, \mathbf{x}) = P(y_1) \prod_{i=2}^{n} P(y_i \mid y_{i-1}) \cdot P(\text{STOP} \mid y_n) \cdot \prod_{i=1}^{n} P(x_i \mid y_i)$$

name	distribution	shape	meaning
initial $\mathbf{s}$	$P(y_1)$	$\lvert\mathcal{T}\rvert$	which tag starts a sentence
transitions T	$P(y_i \mid y_{i-1})$	$\lvert\mathcal{T}\rvert \times (\lvert\mathcal{T}\rvert + 1)$	next tag (incl. STOP) given current tag
emissions E	$P(x_i \mid y_i)$	$\lvert\mathcal{T}\rvert \times \lvert V\rvert$	word **given** tag

Each row is a probability distribution (entries in $[0, 1]$ summing to 1). Transitions don't depend on position i. **Assumptions:** (1) Markov: $y_i \perp y_{1:i-2} \mid y_{i-1}$. (2) Each word depends only on its own tag (so "San Francisco" and "San York" are equally plausible as NNP NNP). **Generative:** models $P(\mathbf{y}, \mathbf{x})$, unlike discriminative LR's $P(y|\mathbf{x})$.

Midterm Fall 2022 Q1(2)

HMM tag transition probabilities are:

Official answer: D. the probability of a POS tag given the preceding tag, not conditioned on the words.

20.2 Parameter estimation: count and normalize

Maximize the generative log-likelihood $\sum_i \log P(\mathbf{y}^{(i)}, \mathbf{x}^{(i)})$. The MLE is relative frequency, like a biased coin: H H H T gives $\hat{p} = 3/4$ (maximizes $3 \log p + \log(1-p)$). No gradient descent needed.

Count and normalize

1. **Initial:** count first tags → divide by the number of sentences.
2. **Transitions:** count tag bigrams including y_n →STOP → divide each row by that tag's outgoing total.
3. **Emissions:** count (tag, word) pairs → divide by the tag's total count. This is $P(\text{word}|\text{tag})$, *not* $P(\text{tag}|\text{word})$.
4. Optional **smoothing**: add pseudo-counts (e.g. +1) so unseen events aren't impossible.

Worked example: lecture: "they can" and "they fish", both N V

Initial: N 2, V 0, so $P(N) = 1$, $P(V) = 0$. Transitions: N→V 2, V→STOP 2, so $P(V|N) = 1$, $P(\text{STOP}|V) = 1$. Emissions: $P(\text{they}|N) = 1$; $P(\text{can}|V) = P(\text{fish}|V) = \frac{1}{2}$ (probability of the *word* given V). **Add-one smoothing on transitions:** N row counts (N 1, V 3, STOP 1), V row (N 1, V 1, STOP 3), giving N: $(\frac{1}{5}, \frac{3}{5}, \frac{1}{5})$ and V: $(\frac{1}{5}, \frac{1}{5}, \frac{3}{5})$. $P(\text{they/N can/V fish/V}) = 1 \cdot 1 \cdot \frac{3}{5} \cdot \frac{1}{2} \cdot \frac{1}{5} \cdot \frac{1}{2} \cdot \frac{3}{5} = \frac{\mathbf{9}}{\mathbf{500}}$ (Exercise 7.2 Q1). Without smoothing the V→V transition would be 0. Exercise 7.2 Q2: no other tag sequence for "they can fish" has nonzero probability (they must be N, can/fish must be V).

20.3 Inference: Viterbi

$\arg\max_{\mathbf{y}} P(\mathbf{y}|\mathbf{x}) = \arg\max_{\mathbf{y}} \frac{P(\mathbf{y},\mathbf{x})}{P(\mathbf{x})} = \arg\max_{\mathbf{y}} \log P(\mathbf{y}, \mathbf{x})$, since $P(\mathbf{x})$ is constant in $\mathbf{y}$.

Viterbi recurrence (log space)

$v_i(y)$ = score of the best path ending in tag y at position i (chart $n \times |\mathcal{T}|$).

$$
\begin{aligned}
v_1(y) &= \log P(y) + \log P(x_1 \mid y) \\
v_i(y) &= \log P(x_i \mid y) + \max_{y'} \big[v_{i-1}(y') + \log P(y \mid y') \big] \quad \text{(store backpointer } y'\text{)} \\
\text{best} &= \max_{y} \big[v_n(y) + \log P(\text{STOP} \mid y) \big], \text{ then follow backpointers.}
\end{aligned}
$$

Works with any real-valued scores (CRFs too). Probability space: replace $+$ with $\times$. **Runtime** $O(nk^2)$ for n words and k tags (each of nk cells maxes over k predecessors). It implicitly searches $O(k^n)$ paths. If each tag can be followed by at most l tags, there are $O(kl^{n-1})$ paths. **Why DP works:** the Markov property means a cell needs only the previous column. Keep *all* tags alive at each step, because later words can change which earlier tag is best.

Worked example: Exercise 7.3 (log probabilities)

$\log P(y_1)$: N -1, V -1. Transitions (to N, V, STOP): from N $(-2, -1, -1)$; from V $(-1, -1, -2)$. Emissions (they, can, fish): N $(-1, -3, -1)$; V $(-3, -1, -1)$.

	they	can	fish
N	$-1-1=\mathbf{-2}$	$-3+\max(-2-2,-4-1)=\mathbf{-7}$ (from N)	$-1+\max(-7-2,-4-1)=\mathbf{-6}$ (from V)
V	$-1-3=\mathbf{-4}$	$-1+\max(-2-1,-4-1)=\mathbf{-4}$ (from N)	$-1+\max(-7-1,-4-1)=\mathbf{-6}$ (from V)

STOP: N $-6-1=-7$; V $-6-2=-8$. Best score

Answer: -7; backpointers N←V←N give

Answer: N V N.

Midterm Fall 2022 Q1(8)–(10)

Initial J 2/3, N 1/3. Transitions (to J, N, STOP): from J $(1/2, 1/4, 1/4)$; from N $(0, 1/2, 1/2)$. Emissions: J: big 1/2, red 1/4, convertible 1/4, car 0; N: big 0, red 0, convertible 1/3, car 2/3.
(8) $P(\text{J/big N/car}) = \frac{2}{3}\cdot\frac{1}{2}\cdot\frac{1}{4}\cdot\frac{2}{3}\cdot\frac{1}{2} =$

Official answer: 1/36.
(9) Tag sequences with nonzero probability for "big convertible car": big must be J and car must be N, so J J N and J N N:

Official answer: 2.
(10) "big car convertible": J N ?; N→J is 0, so only J N N:

Official answer: 1.
Viterbi on (9) (probability space): v_1: J 1/3, N 0. v_2: J $= \frac{1}{3}\cdot\frac{1}{2}\cdot\frac{1}{4} = \frac{1}{24}$; N $= \frac{1}{3}\cdot\frac{1}{4}\cdot\frac{1}{3} = \frac{1}{36}$. v_3: N $= \frac{2}{3}\max(\frac{1}{24}\cdot\frac{1}{4}, \frac{1}{36}\cdot\frac{1}{2}) = \frac{2}{3}\cdot\frac{1}{72} = \frac{1}{108}$ (from N). With STOP, $\frac{1}{216}$. Best: **J N N** ($1/216 > 1/288$ for J J N).

Midterm Fall 2023 Q1(19)–(21)

Viterbi runtime:

Official answer: $O(nk^2)$. Paths searched:

Official answer: $O(k^n)$. If each tag can only be followed by l tags:

Official answer: $O(kl^{n-1})$.

20.4 Full derivations

A. The coin MLE (lecture), by calculus

1. Data H H H T: likelihood $p^3(1-p)$, log-likelihood $\ell(p) = 3\log p + \log(1-p)$.
2. $\ell'(p) = \dfrac{3}{p} - \dfrac{1}{1-p} = 0 \Rightarrow 3(1-p) = p \Rightarrow p = \frac{3}{4}$. $\ell'' < 0$, so it's a maximum.
3. The HMM log-likelihood splits into separate sums for initial, transition and emission parameters, and each row is its own "die". The general Lagrange argument (Topic 12) gives count/total for every row.

B. Why Viterbi is correct (the max-pushing argument)

1. Write the score as a sum of local terms: $\log P(\mathbf{y}, \mathbf{x}) = \big[\log P(y_1) + \log P(x_1|y_1)\big] + \sum_{i=2}^{n}\big[\log P(y_i|y_{i-1}) + \log P(x_i|y_i)\big] + \log P(\text{STOP}|y_n)$.

2. y_1 appears only in the first bracket and in the $i = 2$ transition. So $\max_{y_1}$ can be pushed inside: $\max_{y_1}\left[v_1(y_1) + \log P(y_2|y_1)\right]$, which is a function of y_2 only.
3. Adding $\log P(x_2|y_2)$ gives exactly $v_2(y_2)$. Repeat for $y_2, y_3, \ldots$: each step eliminates one variable, leaving a table indexed by the next tag.
4. This works only because of the Markov property: each tag touches just its neighbor. With transitions to tags two back you'd need pairs of tags as states (the trigram HMM trick).
5. **Runtime:** n columns $\times$ k cells $\times$ a max over k predecessors $= O(nk^2)$. **Paths:** k choices for y_1 and k for each later tag gives k^n; if each tag has at most l successors: $k \cdot l^{n-1}$.

Worked example: the lecture's chart, "they can can fish" (Exercise 7.3 parameters; checked by brute force over all 16 sequences)

	they	can	can	fish
N	$\mathbf{-2}$	$-3 + \max(-2-2, -4-1) = -7$ (N)	$-3 + \max(-7-2, -4-1) = -8$ (V)	$-1 + \max(-8-2, -6-1) = -8$ (V)
V	-4	$-1 + \max(-2-1, -4-1) = \mathbf{-4}$ (N)	$-1 + \max(-7-1, -4-1) = \mathbf{-6}$ (V)	$-1 + \max(-8-1, -6-1) = -8$ (V)

STOP: N $-8-1 = \mathbf{-9}$, V $-8-2 = -10$. Backtrack: N $\leftarrow$ V $\leftarrow$ V $\leftarrow$ N, i.e. **they/N can/V can/V fish/N** ("they can [are able to] can fish"). Runner-up N V V V at -10. The lecture filled the first two columns (the -7 and -4 cells above); note that -7 for the first *can* as N had to be kept, since a later word could have made it best.

Conceptual checks (the style the TA flagged)

- *What does $v_i(y)$ mean?* The log-probability of the best tag prefix $y_1..y_i$ with $y_i = y$, including emissions of $x_1..x_i$. Not a probability of y alone, and not normalized.
- *Why not take the best tag at each column?* That's greedy; the transition to the next word can make a lower-scoring cell part of the best full path (*can* as N above).
- *Why is* $\arg\max P(\mathbf{y}|\mathbf{x}) = \arg\max P(\mathbf{y}, \mathbf{x})$*?* $P(\mathbf{y}|\mathbf{x}) = P(\mathbf{y}, \mathbf{x})/P(\mathbf{x})$ and $P(\mathbf{x})$ doesn't depend on $\mathbf{y}$.
- *Without transitions?* Every position could be maximized independently ($\max_y P(x_i|y)$): no DP needed. Transitions couple the positions.
- *Doubling the number of tags?* Runtime $\times 4$ (k^2). Doubling sentence length: $\times 2$.
- *Sanity check (lecture tip):* for 4–5 words, brute-force all k^n sequences and compare with Viterbi.

20.5 HMMs for POS tagging in practice

A **trigram HMM** uses tag-pair states (y_{i-1}, y_i), so transitions condition on two previous tags and the Viterbi algorithm is unchanged. Trigrams are the sweet spot. About 95% on the Penn Treebank vs. 90% for the most-frequent-tag baseline.

Traps

- Emission direction: $P(\text{word} \mid \text{tag})$, normalized over words for a fixed tag.
- Don't forget the STOP transition at the end, and the initial distribution at the start.
- A single zero factor (unsmoothed) makes the whole sequence probability 0.
- Viterbi takes a *max* (best path); summing instead gives the forward algorithm ($P(\mathbf{x})$, not on

this exam).

- Keep the best score *per tag* at each position; don't greedily commit.

20.6 Practice

Practice

1. Estimate the HMM (initial, transitions incl. STOP, emissions) from: they/N can/V fish/N; fish/N can/V fish/V; they/N fish/V.
2. Using your estimates (no smoothing), compute $P(\text{they/N can/V fish/V})$ and $P(\text{they/N can/V fish/N})$. Which does Viterbi choose?
3. With 45 tags and a 25-word sentence, how many cells does the Viterbi chart have, and roughly how many operations?

Answers

1. Initial: N 1. Transitions: from N: V 3, STOP 1, so $(N\,0, V\,\frac{3}{4}, \text{STOP}\,\frac{1}{4})$. From V: N 1, V 1, STOP 2, so $(\frac{1}{4}, \frac{1}{4}, \frac{1}{2})$. Emissions: N: they $\frac{1}{2}$, fish $\frac{1}{2}$; V: can $\frac{1}{2}$, fish $\frac{1}{2}$.
2. N V V: $1 \cdot \frac{1}{2} \cdot \frac{3}{4} \cdot \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{256}$. N V N: $1 \cdot \frac{1}{2} \cdot \frac{3}{4} \cdot \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{2} \cdot \frac{1}{4} = \frac{3}{512}$. Viterbi picks **N V V**.
3. $25 \times 45 = 1125$ cells; about $25 \times 45^2 \approx 50{,}625$ operations.

TOPIC 21

Constituency Syntax: Trees and Ambiguity

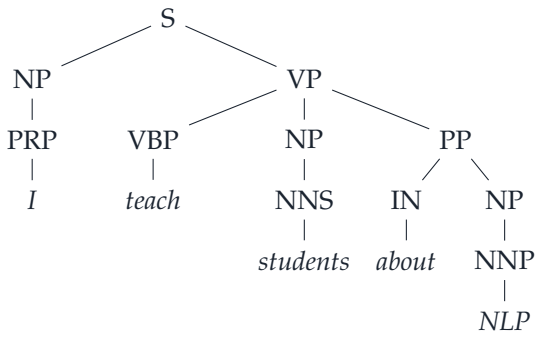
Sources: Lecture 7.4.0 Constituency Parsing; slides seg-28, seg-34; Fall 2023 Q4(b,d); Fall 2022 Q4(a). Dependency parsing is **not** on this exam.

In plain words

Constituency trees group words into nested phrases: noun phrases (NP), verb phrases (VP), prepositional phrases (PP), sentences (S). The leaves are words, the layer above them is POS tags (pre-terminals), and phrases sit above that. The same word string can have different trees with different meanings. Most famously, a PP can attach to a verb or to a noun. The grammar allows both; choosing needs world knowledge, which is why parsing is hard.

21.1 What trees look like

- Root S; phrases NP, VP, PP, ADJP, ADVP, SBAR (sentential complement: "she told me [that I would never amount to anything]"); bottom layer = POS tags.
- Nodes can have any arity: unary (NP → PRP), binary, ternary (VP → VBD NP PP). English is mostly **right-branching**.
- Bracket notation: `(S (NP (PRP I)) (VP (VBP teach) (NP (NNS students)) (PP (IN about) (NP (NNP NLP)))))`.
- Penn Treebank trees also have traces/co-indexation ("a refund that the court estimated *T*-1"), which strictly makes them graphs. We ignore this.
- Constituency suits languages with fairly fixed word order. Free-word-order languages are better served by dependency grammars.



21.2 Ambiguities (must recognize)

type	example and readings
PP attachment	*The children ate the cake with a spoon*: [VP ate the cake][PP with a spoon] (instrument) vs. [NP the cake [PP with a spoon]] (cake that has a spoon, like "cake with icing"). *Eat spaghetti with chopsticks* (VP attachment, instrument) vs. *eat spaghetti with meatballs* (NP attachment). Same tag sequence and same rule types; only the words differ.
PP attachment in headlines	*Ban on nude dancing on Governor's desk*: is the *ban* on the desk, or the *dancing*? [NP ban [PP on nude dancing]] [PP on desk] attaches to ban vs. to dancing.
Modifier scope	*plastic cup holder*: [[plastic cup] holder] (holder for plastic cups) vs. [plastic [cup holder]] (cup holder made of plastic).
Complement structure	*The students complained to the professor that they didn't understand*: relative clause on professor vs. content of the complaint.
Coordination scope	*The man picked up his hammer and saw*: [hammer and saw] (nouns) vs. [picked up his hammer] and [saw] (verb).
POS-driven	*Fed raises interest rates* (Topic 19): different tags give different trees.

PP attachment as trees

VP attachment: (VP (VP (V ate) (NP spaghetti)) (PP (P with) (NP chopsticks))) or (VP V NP PP).
NP attachment: (VP (V ate) (NP (NP spaghetti) (PP (P with) (NP meatballs)))).
A plain PCFG scores these with *rule* probabilities only, so it prefers the same attachment for chopsticks and meatballs (Topic 22). Fixing that needs lexical information.

Conceptual checks

- *Why can't the POS tags alone decide PP attachment? ate spaghetti with chopsticks* and *ate spaghetti with meatballs* have identical tag sequences (V N P N); only the meaning of the last word decides.
- *How many readings?* V NP PP has 2 (PP on the verb or on the noun). V NP PP PP ("saw the man with the telescope on the hill") has 5: each PP attaches to the verb or to some noun to its left, with no crossing branches. The count grows like the Catalan numbers (2, 5, 14, . . .), which is why parsers need a scoring model.
- *What changes between the two trees?* Which constituent the PP is a sister of: a VP-level PP modifies the event; an NP-internal PP modifies the noun.

21.3 Constituency tests (how linguists justify phrases)

- **Substitution by pro-form:** "*They* ate the cake with a spoon" (the children = NP); "the children *did so* with a spoon" (ate the cake = VP).
- **Clefting:** "It was *with a spoon* that the children ate the cake."
- **Answer to a question:** What did they eat? *The cake.* How? *With a spoon.*
- Limits: "She went to and bought food at the store" (*the store* serves both prepositions), so constituency isn't perfectly clean.

Midterm Fall 2022 Q4(a)

Grammar: S→PRP VP; VP→V NP; NP→J N; lexicon I, like/eat, green/red/verde/roja, salsa. Parse "I like green salsa".

Official answer: (S (PRP I) (VP (V like) (NP (J green) (N salsa)))).

Traps

- Write brackets carefully: every node is `(LABEL children...)`; pre-terminals wrap exactly one word.
- Identify which phrase a PP attaches to: a VP-sister PP modifies the verb; an NP-internal PP modifies the noun.
- You don't need encyclopedic grammar. Use the grammar or tags the question gives.

21.4 Practice

Practice

1. Give both bracketed parses of "I saw the man with the telescope" (use S, NP, VP, PP, PRP, V, DT, N, P) and say what each means.
2. For "Ban on nude dancing on Governor's desk", write the NP structure for the reading where the dancing is on the desk.

Answers

1. VP attachment (I used the telescope): (S (NP (PRP I)) (VP (VP (V saw) (NP (DT the) (N man))) (PP (P with) (NP (DT the) (N telescope))))). NP attachment (the man has the telescope): (S (NP (PRP I)) (VP (V saw) (NP (NP (DT the) (N man)) (PP (P with) (NP (DT the) (N telescope)))))).
2. (NP (NP Ban) (PP on (NP (NP nude dancing) (PP on (NP Governor's desk))))). The second PP sits inside the NP headed by *dancing*.

TOPIC 22

PCFGs, Binarization, Markovization, CKY

Sources: Lectures 7.4.1 Probabilistic Context-Free Grammars, 7.4.2 CKY Algorithm, 7.4.3 Refining Grammars; slides seg-34–seg-39; Exercises 7.4, 7.5; Fall 2022 Q4; Fall 2023 Q4.

In plain words

A PCFG is a context-free grammar whose rules have probabilities (for each left-hand symbol, its rules sum to 1). A tree's probability is the product of its rules' probabilities. Get the probabilities from a treebank by **counting and normalizing** rules, after **binarizing** trees so every rule has at most two children. Find the best tree with **CKY**, a dynamic program over spans, $O(n^3|G|)$. Plain PCFGs are too "context-free", so **Markovization** (annotating parents, limiting sibling memory) refines the symbols and boosts accuracy a lot.

22.1 CFG and PCFG

Definitions

CFG $= (N, T, S, R)$: non-terminals N (S, NP, VP, PP, … plus POS tags, which are **pre-terminals**), terminals T (words), start symbol S, rules R. Rules: binary (NP $\rightarrow$ DT NN), unary non-terminal (NP $\rightarrow$ PRP), and lexicon (NN $\rightarrow$ cake).
PCFG: each rule has $P(\text{rule} \mid \text{parent})$, and **probabilities sum to 1 per parent**: $\sum_{\beta} P(A \rightarrow \beta) = 1$.
$P(\text{tree}) = \prod_{\text{rules } r \in \text{tree}} P(r \mid \text{parent}(r))$.

Midterm Fall 2022 Q4(b)–(e)

(b) P((S (PRP I) (VP (V like) (NP (J green) (N salsa))))) $= 1 \cdot 1 \cdot 1 \cdot 1 \cdot 0.5 \cdot 0.25 \cdot 1 =$

Official answer: 1/8 (S, PRP$\rightarrow$I, VP, V$\rightarrow$like 0.5, NP, J$\rightarrow$green 0.25, N$\rightarrow$salsa).
(c) Add one non-terminal rule to parse "I like salsa verde":

Official answer: NP $\rightarrow$ N J.
(d) An ungrammatical sentence it now generates:

Official answer: "I like salsa red".
(e) Split Spanish/English:

Official answer: NP-EN $\rightarrow$ J-EN N-EN; NP-ES $\rightarrow$ N-ES J-ES; J-EN $\rightarrow$ green | red; J-ES $\rightarrow$ verde | roja; N-EN $\rightarrow$ salsa; N-ES $\rightarrow$ salsa (splitting N is optional). You'd also need VP $\rightarrow$ V NP-EN | V NP-ES.

Canvas Exercise 7.4

PCFG: NP$\rightarrow$NP CC NP [0.3]; NP$\rightarrow$NP PP [0.3]; NP$\rightarrow$NNS [0.4]; PP$\rightarrow$P NP [1]; NNS$\rightarrow$cats; CC$\rightarrow$and; P$\rightarrow$in; root NP. "cats and cats":

Answer: 1 parse. "cats and cats in cats":

Answer: 2 parses: [[cats and cats] in cats] vs. [cats and [cats in cats]] (coordination vs. PP attachment scope).

22.2 From treebank to grammar

Pipeline

1. **(Optional) Markovize / annotate** the trees (below).
2. **Binarize** so every rule is binary or unary.
3. **Estimate:** count each rule and normalize by its parent's total. Example: NP→X seen 3 times and NP→Y seen 4 times gives 3/7 and 4/7. This is the MLE (same multinomial logic as HMMs).
4. **Parse** new sentences with CKY.

Midterm Fall 2023 Q4(a) — extract a grammar (no lexicon, no binarization)

Trees: (S (NP (PRP I)) (VP (VBP teach) (NP (NNS students)) (PP (IN about) (NP (NNP NLP))))) and (S (NP (PRP he)) (VP (VBP gives) (NP (NNS lessons)))).

Official answer: S→NP VP; NP→PRP; NP→NNS; NP→NNP; VP→VBP NP PP; VP→VBP NP; PP→IN NP. With counts: S→NP VP 2/2 = 1; NP→PRP 2/5, NP→NNS 2/5, NP→NNP 1/5; VP→VBP NP PP 1/2, VP→VBP NP 1/2; PP→IN NP 1.
(b) Parse "him teach I": (S (NP (PRP him)) (VP (VBP teach) (NP (PRP I)))). **Erratum:** "him" was accidentally missing from the lexicon, so "not parseable" was also accepted. The point: in this grammar the accusative *him* can be a subject, since NPs are interchangeable.

22.3 Binarization

A 4-ary rule like VP → VBD NP PP PP ("sold the book to her for $3") makes estimation sparse and parsing slower (CKY's cost grows with rule arity).

Lossless vs. lossy

Lossless: introduce intermediate symbols that remember what remains: VP → VBD [VP→VBD·NP PP PP], … The first rule carries probability p; the intermediate rules have probability 1, since the symbol already encodes the yield. Equivalent (chain rule) form: generate children one at a time, each conditioned on everything generated so far.
Lossy (e.g. reuse plain VP for intermediates): intermediates compete with real VPs for probability mass, so "VP" learns to spit out PPs. This changes the model.

22.4 Markovization (refining grammars)

PCFGs are **too context-free**. NP → PRP is 21% for subject NPs (under S) but only 4% for object NPs (under VP); overall NP→NP PP is 11%, DT NN 9%, PRP 6%.

Vertical and horizontal Markovization

Vertical (parent annotation, order v): relabel each node with its parent: NP under S becomes **NP^S**, NP under VP becomes **NP^VP**. Applied to the treebank *before* extracting the grammar. Captures context-dependent rule distributions; grammar gets bigger.
Horizontal (order h, during binarization): intermediate symbols remember only the last h generated siblings. $h = 0$ is like lossy (most compact, weakest); $h = 1$ remembers the previous sibling; $h = \infty$ is fully lossless. The head is usually always remembered.

Klein & Manning (2003): going from $v = 1, h = 0$ (≈70 F1) to good v, h settings gives ≈80 F1. Lexicalized parsers (Collins, Charniak) ≈90 F1 annotate each node with its **head word** (VP → verb, NP → noun, S → verb). That fixes attachments that depend on words (dogs in houses and cats), but makes the grammar huge and the parsers very complex.

Midterm Fall 2023 Q4(c)–(e) — vertical Markovization of NPs only

(c) New grammar:

Official answer: S→NP^S VP; NP^S→PRP; NP^VP→NNS; NP^PP→NNP; VP→VBP NP^VP PP; VP→VBP NP^VP; PP→IN NP^PP.

(d) Parse "him teach I"?

Official answer: Not parseable: PRP can only appear under NP^S, and the object position needs NP^VP → NNS.

(e) The language of the Markovized grammar vs. the original:

Official answer: a subset, since NPs can no longer be swapped between subject and object positions.

22.5 CKY

CKY recurrence (log probabilities, binary grammar)

Chart $t(i, j, X)$ = score of the best derivation of symbol X over span $[i, j)$ (fenceposts $0..n$).

$$\textbf{Base: } t(i, i+1, X) = \log P(X \to w_{i+1})$$
$$\textbf{Recurrence: } t(i, j, X) = \max_{k:\, i<k<j} \; \max_{X \to X_1 X_2} \left[\log P(X \to X_1 X_2) + t(i, k, X_1) + t(k, j, X_2)\right]$$

Answer: $t(0, n, S)$; follow backpointers (split k and rule) to recover the tree. $\arg\max_T P(T|\mathbf{x}) = \arg\max_T P(T, \mathbf{x})$. **Runtime** $O(n^3|G|)$: n^2 cells × $O(n)$ split points × grammar constant. Unary rules need an extra closure step (not covered). Fill the chart by increasing span length.

Worked example: lecture CKY: "the child raises it" ($\log_2$: $\log 1 = 0, \log 0.5 = -1$)

Grammar: DT→the, NN→child, NNS→raises, VBZ→raises, PRP→it (all 1.0); S→NP VP 1.0; NP→DT NN 0.5; NP→NN NNS 0.5; VP→VBZ PRP 1.0.

span	entries	how
$[0,1]$ the	DT: 0	lexicon
$[1,2]$ child	NN: 0	
$[2,3]$ raises	NNS: 0, VBZ: 0	two tags
$[3,4]$ it	PRP: 0	
$[0,2]$	NP: -1	NP→DT NN: $-1+0+0$
$[1,3]$	NP: -1	NP→NN NNS (a dead end)
$[2,4]$	VP: 0	VP→VBZ PRP
$[0,3]$, $[1,4]$	—	no rule applies
$[0,4]$	S: -1	split $k=2$: S→NP[0,2] VP[2,4] $= 0-1+0$

Tree: (S (NP (DT the) (NN child)) (VP (VBZ raises) (PRP it))), probability $2^{-1} = 0.5$.

Canvas Exercise 7.5

Same grammar. Best parse of "child raises raises it":

Answer: (S (NP (NN child) (NNS raises)) (VP (VBZ raises) (PRP it))), score -1. "the child raises raises" has no parse. Which single added rule fixes it?

Answer: VP→VBZ NNS, or PRP→raises. (Not NP→NNS VBZ; not S→NP NP.)

Worked example: why a plain PCFG can't resolve PP attachment

Grammar: S→PRP VP 1; VP→V NP 0.7; VP→VP PP 0.3; NP→NP PP 0.2; PP→P NP 1; NP→spaghetti/chopsticks/meatballs (0.32/0.24/0.24). "I ate spaghetti with chopsticks": VP-attach $= 0.3 \cdot 0.7 \cdot 0.32 \cdot 0.24 = 0.016128$; NP-attach $= 0.7 \cdot 0.2 \cdot 0.32 \cdot 0.24 = 0.010752$, so VP-attach wins. "…with meatballs": exactly the same numbers, so VP-attach wins again, which is wrong. Rule probabilities don't see the words, so lexicalization or a discriminative parser is needed.

22.6 Full derivations and conceptual checks

A. Lossless binarization keeps tree probabilities (proof)

1. Original rule VP $\to$ VBD NP PP PP with probability p.
2. Binarized: VP $\to$ VBD @1 with probability p; @1 $\to$ NP @2 with probability 1; @2 $\to$ PP PP with probability 1. (@1, @2 each have exactly one rule, since the symbol name records the remaining children.)
3. Product along the binarized fragment: $p \cdot 1 \cdot 1 = p$, the same as before. Every other rule in the tree is untouched, so every tree probability is unchanged.
4. With $h < \infty$ an intermediate symbol like @VP[NP] is shared by different original rules, so it has several rules, each with probability < 1. The product is no longer the original p: lossy, but less sparse.

B. Why vertical Markovization gives a subset of the language

1. Every annotated rule $A\hat{}P \to B\hat{}A\ C\hat{}A$ maps to an original rule $A \to B\ C$ by erasing annotations. So every annotated tree maps to an original tree with the same words: the new language is $\subseteq$ the old one.
2. The reverse fails: an original derivation may combine pieces seen only in different contexts (PRP as an object NP in Fall 2023 Q4), and the annotated grammar has no rule for that. Hence "him teach I" is no longer parseable.

C. CKY runtime, counted

1. Spans $[i, j)$ with $0 \le i < j \le n$: $\binom{n+1}{2} = \frac{n(n+1)}{2} = O(n^2)$ cells.
2. A span of length ℓ has $\ell - 1$ split points. Total (span, split) pairs: $\sum_{\ell=2}^{n}(n-\ell+1)(\ell-1) = \frac{n^3-n}{6} = O(n^3)$. Check $n = 4$: $3 \cdot 1 + 2 \cdot 2 + 1 \cdot 3 = 10 = \frac{64-4}{6}$.
3. For each pair, loop over binary rules: $\times|G|$, giving $O(n^3|G|)$.
4. With rules of arity r you'd choose $r - 1$ split points per span: $O(n^{r+1})$. That's why we binarize.

Conceptual checks for CKY/PCFGs

- *What does $t(i, j, X)$ hold?* The best log-probability of any subtree rooted in X whose yield is exactly words $i+1..j$. $-\infty$ means X can't produce that span.
- *Why fill short spans first?* The recurrence for $[i, j)$ uses $[i, k)$ and $[k, j)$, which are strictly shorter.
- *CKY vs. Viterbi.* Viterbi: cells (position, tag), max over the previous tag, $O(nk^2)$. CKY: cells (span, symbol), max over split and rule, $O(n^3|G|)$. Both: max of (child scores + local log-probability), then backpointers.
- *Is $t(0, n, S) = -\infty$ possible?* Yes, when the grammar can't generate the sentence (Exercise 7.5).
- *Why does the PP-attachment example fail?* Both trees use the same lexicon rules; only the rule *shapes* differ, and those have fixed probabilities regardless of *chopsticks* vs. *meatballs*.
- *What does Markovization change at parse time?* Nothing algorithmic: CKY runs on the bigger, annotated grammar ($|G|$ grows), then annotations are stripped from the output tree.

Traps

- Rule probabilities normalize over rules with the *same parent*, not over all rules.
- Tree probability = product over *all* rules, including lexicon rules (e.g. V→like 0.5).
- Grammar-extraction questions: follow the requested order (S, NP, VP, PP), exclude the lexicon if told, and list each distinct rule once.
- Vertical Markovization makes a grammar that generates a *subset* of the original (more specific symbols).
- CKY needs a binarized grammar; spans use fenceposts; split point k is strictly between i and j.
- Log base: use the one stated (the lecture CKY uses $\log_2$).

22.7 Practice

Practice

1. Losslessly binarize VP → VBD NP PP PP with $h = \infty$, then with $h = 1$ (write the intermediate symbols).
2. Apply vertical Markovization ($v = 2$, all non-pre-terminal nodes) to (S (NP (DT the) (NN dog)) (VP (VBD saw) (NP (PRP me)))). List the extracted non-lexical rules.
3. How many parses does "cats and cats and cats" have under Exercise 7.4's grammar?

Answers

1. $h = \infty$: VP→VBD @VP[VBD]; @VP[VBD]→NP @VP[VBD,NP]; @VP[VBD,NP]→PP PP (each intermediate remembers all previous siblings). $h = 1$: VP→VBD @VP[VBD]; @VP[VBD]→NP @VP[NP]; @VP[NP]→PP PP. Only the last sibling is remembered, so @VP[NP] is shared with other VPs that just generated an NP.
2. S→NP^S VP^S; NP^S→DT NN; VP^S→VBD NP^VP; NP^VP→PRP.
3. 2: [[cats and cats] and cats] and [cats and [cats and cats]] (NP→NP CC NP applies at two levels).